

# EN3021 Individual Project - Progress as of OCT03

210609M Silva M.K.Y.U.N.

October 10, 2024

**Current Status:** 60% complete

## Progress this week

Following parts of the project were done for the week OCT03 - OCT10:

- Corrected Adder/Subtractor to include normalisation
- Did the HDL design for the adder subtractor part and tested using ModelSim
- Started the paper design for the multiplier part

## Correction: Normalisation stage in the Adder/Subtractor

One of the errors that were done in the paper design was not including a normalisation stage. This is crucial to retain the IEEE 754 notation, as the mantissa of the number is such that the MSB 1 is ignored; i.e.:

- If the result was  $0.0001010 \dots \times 2^x$ , writing the mantissa as 0001010... and exponent as  $x$  would be wrong
- It needs to be normalised to  $1.010 \dots \times 2^{(x-4)}$ , the mantissa needs to be 010... and the exponent needs to be  $(x - 4)$

## HDL design for Adder and Subtractor part

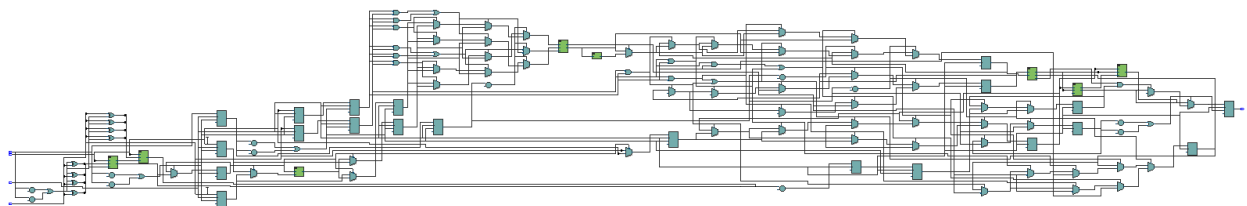


Figure 1: RTL view of the entire Adder/Subtractor part

In the updated design there are 4 stages:

- Stage1: Exponent difference
- Stage2: Shifting
- Stage3: ALU Operation and Sign Resolution
- Stage4: Normalisation

## Carry Look Ahead Adder

Here, to reduce the time taken for additions, I used carry look-ahead adders. Since even the smallest number of bits needed to be added is 8, CLAs were used in almost all the places.

```
1 module cla(  
2     input  [3:0] A, B,  
3     input  CIN,  
4     output COUT,  
5     output [3:0] S  
6 );  
7  
8 // For carry calculation  
9  
10 wire [3:0] P, G;  
11 assign P = A ^ B;  
12 assign G = A & B;  
13  
14 wire [3:0] CARRY;  
15  
16  
17  
18 assign CARRY[0] = G[0] | (P[0] & CIN);  
19 // CARRY[0]  
19 assign CARRY[1] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & CIN);  
20 // CARRY[1]  
20 assign CARRY[2] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & CIN);  
21 // CARRY[2]  
21 assign CARRY[3] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G  
22 [0]) | (P[3] & P[2] & P[1] & P[0] & CIN); // CARRY[3]  
23  
24 assign COUT = CARRY[3];  
25  
26 // Adder  
27 assign S = P ^ {CARRY[2:0], CIN};  
28  
29 endmodule
```

## Main Adder/Subtractor

Here a collection of CLAs were put into a separate module to declutter the code and ease of reading.

```
1 module cla_add(  
2     input  [23:0] A,B,  
3     input  CIN,  
4     output [23:0] OUT,  
5     output OF  
6 );  
7  
8 wire COUT0, COUT1, COUT2, COUT3, COUT4, COUT5;  
9  
10 cla claalu_0(.A(A[ 3: 0]), .B(B[ 3: 0]~{4{CIN}}), .CIN(CIN), .COUT(COUT0), .S(OUT[3:0]))  
11 ;  
11 cla claalu_1(.A(A[ 7: 4]), .B(B[ 7: 4]~{4{CIN}}), .CIN(COUT0), .COUT(COUT1), .S(OUT[7:4]))  
12 ;  
12 cla claalu_2(.A(A[11: 8]), .B(B[11: 8]~{4{CIN}}), .CIN(COUT1), .COUT(COUT2), .S(OUT[11:8]))  
13 ;  
13 cla claalu_3(.A(A[15:12]), .B(B[15:12]~{4{CIN}}), .CIN(COUT2), .COUT(COUT3), .S(OUT  
14 [15:12]));  
14 cla claalu_4(.A(A[19:16]), .B(B[19:16]~{4{CIN}}), .CIN(COUT3), .COUT(COUT4), .S(OUT  
[19:16]));
```

```

15   cla claalu_5(.A(A[23:20]), .B(B[23:20]~{4{CIN}}), .CIN(COUT4), .COUT(COUT5), .S(OUT
    [23:20]));
16
17   assign OF = COUT5;
18
19 endmodule

```

## Two's Complement Converter

Here is a modified CLA, where one of the inputs is removed, hence using fewer resources. This is used in sign resolution and the shifter.

```

1 module twos_cla(
2     input  [3:0] B,
3     input  CIN,
4     output COUT,
5     output [3:0] S
6 );
7
8 // For carry calculation
9
10 wire [3:0] CARRY;
11
12
13
14 assign CARRY[0] = (B[0] & CIN);           // CARRY[0]
15 assign CARRY[1] = (B[1] & B[0] & CIN);     // CARRY[1]
16 assign CARRY[2] = (B[2] & B[1] & B[0] & CIN); // CARRY[2]
17 assign CARRY[3] = (B[3] & B[2] & B[1] & B[0] & CIN); // CARRY[3]
18
19
20 assign COUT = CARRY[3];
21
22 // Adder
23 assign S = (B ^ {CARRY[2:0], CIN});
24
25 endmodule

```

```

1 module twoscomp(
2     input  [23:0] B,
3     output [23:0] OUT
4 );
5
6 wire COUT0, COUT1, COUT2, COUT3, COUT4;
7
8 twos_cla t0(.B(~B[ 3: 0]), .CIN(1'b1), .COUT(COUT0), .S(OUT[3:0]));
9 twos_cla t1(.B(~B[ 7: 4]), .CIN(COUT0), .COUT(COUT1), .S(OUT[7:4]));
10 twos_cla t2(.B(~B[11: 8]), .CIN(COUT1), .COUT(COUT2), .S(OUT[11:8]));
11 twos_cla t3(.B(~B[15:12]), .CIN(COUT2), .COUT(COUT3), .S(OUT[15:12]));
12 twos_cla t4(.B(~B[19:16]), .CIN(COUT3), .COUT(COUT4), .S(OUT[19:16]));
13 twos_cla t5(.B(~B[23:20]), .CIN(COUT4), .COUT(), .S(OUT[23:20]));
14
15 endmodule

```

## Shifter

The shifter is separated as a module, as the number with the lower exponent needs to be shifted right as per the design, by the absolute value of the difference of the exponents.

```

1 module shifter(

```

```

2   input  [23:0] IN,
3   input  [7:0] BY,
4   output reg [23:0] OUT
5 );
6
7   wire COUT_INT;
8   wire [7:0] BY_COMPL;
9
10  // Calculating the absolute
11
12  twos_cla t_shft0(
13    .B(~BY[3:0]),
14    .CIN(1'b1),
15    .COUT(COUT_INT),
16    .S(BY_COMPL[3:0])
17  );
18
19  twos_cla t_shft1(
20    .B(~BY[7:4]),
21    .CIN(COUT_INT),
22    .COUT(),
23    .S(BY_COMPL[7:4])
24  );
25
26  always@(*) begin
27    if (BY[7] == 1'b0) begin // When BY is > 0
28      OUT = IN >> BY;
29    end else begin // When BY is < 0
30      OUT = IN >> BY_COMPL;
31    end
32  end
33 endmodule

```

## Normaliser

This normalises the number we get as a result of the ALU. Here it checks looks for the first 1 of the result and adjusts the mantissa such that the bit after the first 1 is the MSB of the mantissa part, and adjusts the exponent accordingly.

```

1 module normal(
2   input  [23:0] IN,
3   input  INOF,
4   output reg [22:0] OUT,
5   output reg [7:0] COUNT,
6   output reg ZEROFLAG
7 );
8
9 always@(*) begin
10   if (INOF == 1'b1) begin
11     OUT <= IN[23:1]; COUNT <= 8'b1; ZEROFLAG <= 1'b0;
12   end else begin
13     casez (IN)
14       24'b1?????????????????????:
15       begin
16         OUT <= IN[22:0]; COUNT <= 8'd0; ZEROFLAG <= 1'b0;
17       end
18
19       24'b01?????????????????????:
20       begin
21         OUT <= {IN[21:0], 1'b0}; COUNT <= -8'd1; ZEROFLAG <= 1'b0;
22       end
23
24       24'b001?????????????????????:
25       begin

```

```

26         OUT <= {IN[20:0], 2'b0}; COUNT <= -8'd2; ZEROFLAG <= 1'b0;
27     end
28
29     24'b0001?????????????????:
30     begin
31         OUT <= {IN[19:0], 3'b0}; COUNT <= -8'd3; ZEROFLAG <= 1'b0;
32     end
33
34     24'b00001?????????????????:
35     begin
36         OUT <= {IN[18:0], 4'b0}; COUNT <= -8'd4; ZEROFLAG <= 1'b0;
37     end
38
39     24'b000001?????????????????:
40     begin
41         OUT <= {IN[17:0], 5'b0}; COUNT <= -8'd5; ZEROFLAG <= 1'b0;
42     end
43
44     24'b0000001?????????????????:
45     begin
46         OUT <= {IN[16:0], 6'b0}; COUNT <= -8'd6; ZEROFLAG <= 1'b0;
47     end
48
49     24'b00000001?????????????????:
50     begin
51         OUT <= {IN[15:0], 7'b0}; COUNT <= -8'd7; ZEROFLAG <= 1'b0;
52     end
53
54     24'b000000001?????????????????:
55     begin
56         OUT <= {IN[14:0], 8'b0}; COUNT <= -8'd8; ZEROFLAG <= 1'b0;
57     end
58
59     24'b0000000001?????????????????:
60     begin
61         OUT <= {IN[13:0], 9'b0}; COUNT <= -8'd9; ZEROFLAG <= 1'b0;
62     end
63
64     24'b00000000001?????????????????:
65     begin
66         OUT <= {IN[12:0], 10'b0}; COUNT <= -8'd10; ZEROFLAG <= 1'b0;
67     end
68
69     24'b000000000001?????????????????:
70     begin
71         OUT <= {IN[11:0], 11'b0}; COUNT <= -8'd11; ZEROFLAG <= 1'b0;
72     end
73
74     24'b0000000000001?????????????????:
75     begin
76         OUT <= {IN[10:0], 12'b0}; COUNT <= -8'd12; ZEROFLAG <= 1'b0;
77     end
78
79     24'b00000000000001?????????????????:
80     begin
81         OUT <= {IN[9:0], 13'b0}; COUNT <= -8'd13; ZEROFLAG <= 1'b0;
82     end
83
84     24'b000000000000001?????????????????:
85     begin
86         OUT <= {IN[8:0], 14'b0}; COUNT <= -8'd14; ZEROFLAG <= 1'b0;
87     end
88
89     24'b0000000000000001?????????????????:
90     begin
91         OUT <= {IN[7:0], 15'b0}; COUNT <= -8'd15; ZEROFLAG <= 1'b0;
92     end
93

```

```

94      24'b00000000000000000001??????:
95      begin
96          OUT <= {IN[6:0], 16'b0}; COUNT <= -8'd16; ZEROFLAG <= 1'b0;
97      end
98
99      24'b00000000000000000001??????:
100     begin
101         OUT <= {IN[5:0], 17'b0}; COUNT <= -8'd17; ZEROFLAG <= 1'b0;
102     end
103
104     24'b00000000000000000001??????:
105     begin
106         OUT <= {IN[4:0], 18'b0}; COUNT <= -8'd18; ZEROFLAG <= 1'b0;
107     end
108
109     24'b00000000000000000001?????:
110     begin
111         OUT <= {IN[3:0], 19'b0}; COUNT <= -8'd19; ZEROFLAG <= 1'b0;
112     end
113
114     24'b00000000000000000001???:
115     begin
116         OUT <= {IN[2:0], 20'b0}; COUNT <= -8'd20; ZEROFLAG <= 1'b0;
117     end
118
119     24'b00000000000000000001??:
120     begin
121         OUT <= {IN[1:0], 21'b0}; COUNT <= -8'd21; ZEROFLAG <= 1'b0;
122     end
123
124     24'b00000000000000000001?:
125     begin
126         OUT <= {IN[0], 22'b0}; COUNT <= -8'd22; ZEROFLAG <= 1'b0;
127     end
128
129     24'b00000000000000000000001:
130     begin
131         OUT <= {23'b0}; COUNT <= -8'd23; ZEROFLAG <= 1'b0;
132     end
133
134     default:
135     begin
136         OUT <= {23'b0}; COUNT <= -8'd24; ZEROFLAG <= 1'b1;
137     end
138 endcase
139 end
140 end
141 endmodule

```

## Integration of these modules

Above modules have been integrated to make up the Adder/subtractor part as below:

```

1 // IEEE754 Format
2 //
3 // +-----+-----+-----+
4 // | 31 | [30:23] | [22:0] |
5 // | Sign | Exponent | Mantissa |
6 // +-----+-----+-----+
7
8
9 module alu(
10     input FPUCLK,
11     input [31:0] A, B,
12     input [2:0] CTRL,

```

```

13     output reg [31:0] OUT
14 );
15
16
17 // Pipeline registers
18 reg [23:0] APP0, APP1, BPP0, BPP1, RESM; // Pipeline registers for number and cmd
19 reg [31:0] OUT0;
20 reg ASIGN0, ASIGN1, BSIGN0, BSIGN1, SIGN, OF0;
21 reg [2:0] CTRL1, CTRL2, CTRL3;
22 reg [7:0] EXPONENT;
23
24
25 // For exponents
26 reg EXPCIN; // Add or subtract the exponents
27 wire [7:0] EXPDIF_w; // Result of exponents (for pipeline stage1)
28 wire COUT_EX;
29 reg [7:0] EXPCARRY0, EXPCARRY1; // Exp of greater to be carried
30 reg EXPZFLAG0, EXPZFLAG1, EXPZFLAG2;
31
32 cla claexp_0(.A(A[26:23]), .B(B[26:23]~{4{EXPCIN}}), .CIN(EXPCIN), .COUT(COUT_EX), .S(
33   EXPDIF_w[3:0]));
34 cla claexp_1(.A(A[30:27]), .B(B[30:27]~{4{EXPCIN}}), .CIN(COUT_EX), .COUT(), .S(
35   EXPDIF_w[7:4]));
36
37 reg [7:0] EXPDIF0;
38
39 // For shifter
40 reg [23:0] TBSHIFTED;
41 wire [23:0] SHIFTOUT_w;
42
43 shifter shift_0(.IN(TBSHIFTED), .BY(EXPDIF0), .OUT(SHIFTOUT_w));
44
45 // For operations
46 reg [23:0] A_ALUIN, B_ALUIN;
47 wire [23:0] ALUOUT;
48 reg ALUCIN;
49 wire OVERFLOW;
50
51 cla_add cla_add0(
52   .A(A_ALUIN),
53   .B(B_ALUIN),
54   .CIN(ALUCIN),
55   .OUT(ALUOUT),
56   .OF(OVERFLOW)
57 );
58
59 // For sign resolution
60 wire [23:0] TWOSOUT;
61
62 twoscomp twosconv0(
63   .B(ALUOUT),
64   .OUT(TWOSOUT)
65 );
66
67 // Normalising
68 wire [22:0] NORMOUT;
69 wire NORMFLAG, NORMZERO;
70 wire [7:0] NORM_SHIFT;
71
72 normal normalize0(
73   .IN(RESM),
74   .INOF(OF0),
75   .OUT(NORMOUT),
76   .COUNT(NORM_SHIFT),
77   .ZEROFLAG(NORMZERO)
78 );

```

```

79
80 // Signres
81 wire COUT_SRES;
82 wire [7:0] EXPSRES;
83
84 cla clasresexp_0(.A(EXPCARRY1[3:0]), .B(NORM_SHIFT[3:0]), .CIN(1'b0), .COUT(
85   COUT_SRES), .S(EXPSRES[3:0]));
86
87 cla clasresexp_1(.A(EXPCARRY1[7:4]), .B(NORM_SHIFT[7:4]), .CIN(COUT_SRES), .COUT(),
88   .S(EXPSRES[7:4]));
89
90 // codes
91 parameter ADD = 3'b000, SUB = 3'b001;
92
93 // Combinational parts of each stage
94
95 always@(*) begin
96   // -----
97   // Stage 1 : Exponent calculation
98   // -----
99   if (CTRL == ADD | CTRL == SUB) begin
100     EXPCIN <= 1'b1;
101   end else begin
102     EXPCIN <= 1'b0;
103   end
104
105   // -----
106   // Stage 2 : Shifting
107   // -----
108   if (CTRL1 == ADD | CTRL1 == SUB) begin
109     // Deciding which to be shifted
110     if (EXPDIFO[7] == 1'b0) begin // exp(A) > exp(B), shift B to right
111       TBSHIFTED <= BPP0;
112     end else begin // exp(B) > exp(A), shift A to right
113       TBSHIFTED <= APP0;
114     end
115   end else begin
116     TBSHIFTED <= 24'bX;
117   end
118
119   // -----
120   // Stage 3 : Operation
121   // -----
122   if (CTRL2 == ADD) begin
123     if (((ASIGN1 == 1'b0) & (BSIGN1 == 1'b0)) | ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b1)))
124       begin // if signs are the same
125         A_ALUIN <= APP1;
126         B_ALUIN <= BPP1;
127         ALUCIN <= 1'b0;
128       end else if ((ASIGN1 == 1'b0) & (BSIGN1 == 1'b1)) begin
129         A_ALUIN <= APP1; // (+A) + (-B) = (A-B)
130         B_ALUIN <= BPP1;
131         ALUCIN <= 1'b1;
132       end else if ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)) begin
133         A_ALUIN <= BPP1; // Flipping (-A)+(+B) = (B-A)
134         B_ALUIN <= APP1;
135         ALUCIN <= 1'b1;
136       end else begin
137         A_ALUIN <= 24'bX;
138         B_ALUIN <= 24'bX;
139         ALUCIN <= 1'bX;
140       end
141   end else if (CTRL2 == SUB) begin
142     if (((ASIGN1 == 1'b0) & (BSIGN1 == 1'b1)) | ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)))
143       begin // if signs are different
144         A_ALUIN <= APP1;

```



```

143     B_ALUIN <= BPP1;
144     ALUCIN  <= 1'b0;
145     end else if ((ASIGN1 == 1'b0) & (BSIGN1 == 1'b0)) begin
146         A_ALUIN <= APP1; // (+A) - (+B) = (A-B)
147         B_ALUIN <= BPP1;
148         ALUCIN  <= 1'b1;
149     end else if ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)) begin
150         A_ALUIN <= BPP1; // Flipping (-A)-(-B) = (B-A)
151         B_ALUIN <= APP1;
152         ALUCIN  <= 1'b1;
153     end else begin
154         A_ALUIN <= 24'bX;
155         B_ALUIN <= 24'bX;
156         ALUCIN  <= 1'bX;
157     end
158 end else begin
159     A_ALUIN <= 24'bX;
160     B_ALUIN <= 24'bX;
161     ALUCIN  <= 1'bX;
162 end
163
164 // -----
165 // Stage 4 : Sign resolution
166 // -----
167 if ((CTRL3 == ADD)|(CTRL3 == SUB)) begin
168     if ((EXPZFLAG2 == 1'b1) & (NORMZERO == 1'b1)) begin
169         EXPONENT <= 8'b0;
170     end else begin
171         EXPONENT <= EXPSRES;
172     end
173 end else begin
174     EXPONENT <= EXPSRES; // TO BE CHANGED!!!
175 end
176 end
177
178
179 // Let ADD be 3'b000
180 always@(posedge FPUCCLK) begin
181     // -----
182     // Stage 1 : Exponent calculation
183     // -----
184
185     if (CTRL == ADD | CTRL == SUB) begin
186         APPO <= {1'b1,A[22:0]};
187         BPPO <= {1'b1,B[22:0]};
188         ASIGN0 <= A[31];
189         BSIGN0 <= B[31];
190
191         EXPDIFO <= EXPDIF_w;
192         CTRL1 <= CTRL;
193     end
194
195     if (EXPDIF_w == 8'b0) begin
196         EXPZFLAG0 <= 1'b1;
197     end else begin
198         EXPZFLAG0 <= 1'b0;
199     end
200
201
202 // -----
203 // Stage 2 : Shifting
204 // -----
205 if (CTRL1 == ADD | CTRL1 == SUB) begin
206     EXPZFLAG1 <= EXPZFLAG0;
207     CTRL2 <= CTRL1;
208     ASIGN1 <= ASIGN0;
209     BSIGN1 <= BSIGN0;
210

```

```

211 // Deciding which to be shifted
212 if (EXPDIPO[7] == 1'b0) begin // exp(A) > exp(B), shift B to right
213     APP1 <= APP0;
214     BPP1 <= SHIFTOUT_w;
215     EXPCARRY0 <= A[30:23];
216 end else begin // exp(B) > exp(A), shift A to right
217     APP1 <= SHIFTOUT_w;
218     BPP1 <= BPP0;
219     EXPCARRY0 <= B[30:23];
220 end
221 end
222
223
224 // -----
225 // Stage 3 : Operation
226 // -----
227 if (CTRL2 == ADD) begin
228     EXPZFLAG2 <= EXPZFLAG1;
229     CTRL3 <= CTRL2;
230
231     if (((ASIGN1 == 1'b0) & (BSIGN1 == 1'b0)) | ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b1)))
232     begin // if signs are the same
233         RESM <= ALUOUT;
234         OF0 <= OVERFLOW;
235         EXPCARRY1 <= EXPCARRY0;
236         SIGN <= ASIGN1;
237
238     end else if ((ASIGN1 == 1'b0) & (BSIGN1 == 1'b1)) begin
239
240         if (OVERFLOW == 1'b0) begin
241             RESM <= TWOSOUT;
242         end else begin
243             RESM <= ALUOUT;
244         end
245
246         OF0 <= 1'b0;
247         EXPCARRY1 <= EXPCARRY0;
248         SIGN <= ~OVERFLOW;
249
250     end else if ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)) begin
251
252         if (OVERFLOW == 1'b0) begin
253             RESM <= TWOSOUT;
254         end else begin
255             RESM <= ALUOUT;
256         end
257
258         OF0 <= 1'b0;
259         EXPCARRY1 <= EXPCARRY0;
260         SIGN <= ~OVERFLOW;
261     end
262
263 end else if (CTRL2 == SUB) begin
264     CTRL3 <= CTRL2;
265     EXPZFLAG2 <= EXPZFLAG1;
266
267     if (((ASIGN1 == 1'b0) & (BSIGN1 == 1'b1)) | ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)))
268     begin // if signs are different
269         RESM <= ALUOUT;
270         OF0 <= OVERFLOW;
271         EXPCARRY1 <= EXPCARRY0;
272         SIGN <= ASIGN1;
273
274     end else if ((ASIGN1 == 1'b0) & (BSIGN1 == 1'b0)) begin
275         if (OVERFLOW == 1'b0) begin
276             RESM <= TWOSOUT;
277         end else begin
278             RESM <= ALUOUT;
279         end
280     end else if ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b1)) begin
281         if (OVERFLOW == 1'b0) begin
282             RESM <= TWOSOUT;
283         end else begin
284             RESM <= ALUOUT;
285         end
286     end
287 end

```

```

277     end
278
279     OF0 <= 1'b0;
280     EXPCARRY1 <= EXPCARRY0;
281     SIGN <= ~OVERFLOW;
282
283     end else if ((ASIGN1 == 1'b1) & (BSIGN1 == 1'b0)) begin
284         if (OVERFLOW == 1'b0) begin
285             RESM <= TWOSOUT;
286         end else begin
287             RESM <= ALUOUT;
288         end
289
290         OF0 <= 1'b0;
291         EXPCARRY1 <= EXPCARRY0;
292         SIGN <= ~OVERFLOW;
293     end
294 end
295
296
297 // -----
298 // Stage 4 : Sign resolution
299 // -----
300 if ((CTRL3 == ADD)|(CTRL3 == SUB)) begin
301     OUT <= {SIGN, EXPONENT, NORMOUT};
302 end
303 end
304 endmodule

```

Here the resulting ALU can perform addition or subtraction on IEEE 754 single precision floating point numbers within 4 clock cycles. Here the clock frequency this tested under was 50MHz. The design uses 185 registers, and 803 logic elements, which is under 1% of the 114,480 of the EP4CE115F29C7 chip on the DE2-115 board.

## ModelSim Simulation

Here in the following example, I have added 160.125 and -3125.3125, and I have got the result 2965.1875 (truncated by Modelsim as 2965.19) 4 clock cycles later.

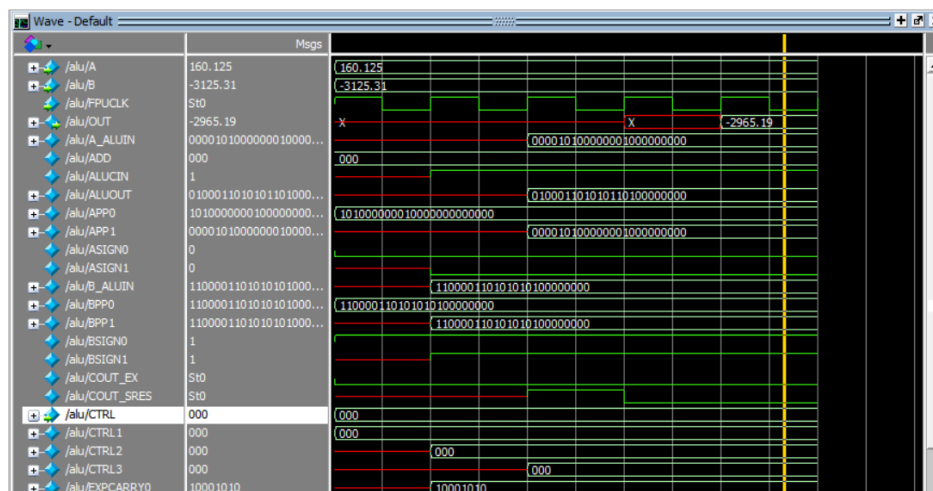


Figure 2: ModelSim Simulation (Here the important signals were moved to the top for visualization purposes)

## Multiplier - Paper Design

For the multiplication of the two IEEE754 floating point numbers, I planned on using Booth's Algorithm with parallel addition. I got the parallel addition part from the Wallace tree design. This addition is done in 5 stages as below:

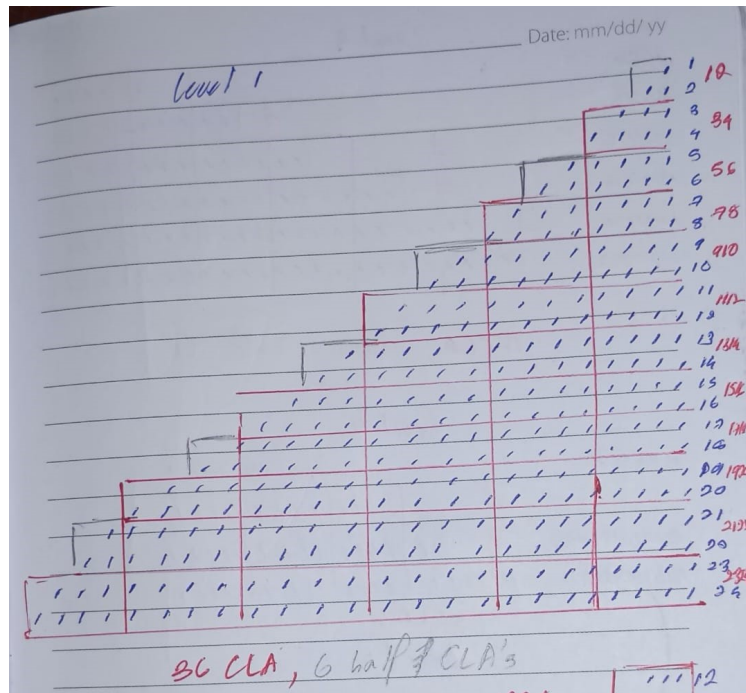


Figure 3: Level 1 - Adding consequently shifted versions where A is shifted left each time, depending on the corresponding bit of B

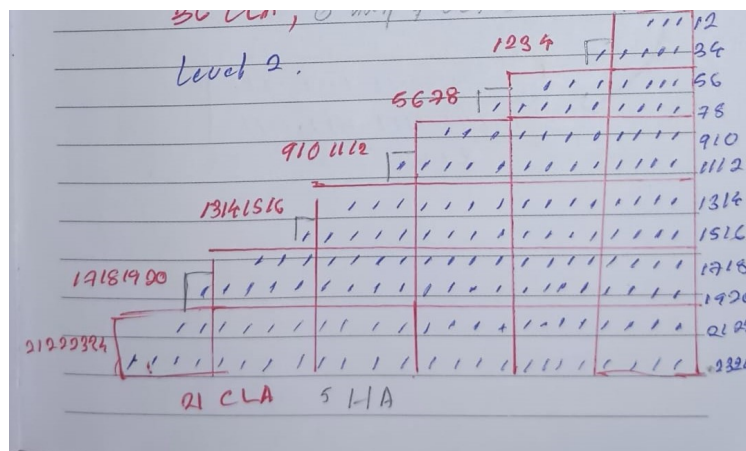


Figure 4: Level 2 - Adding consequently shifted versions of level 1 results

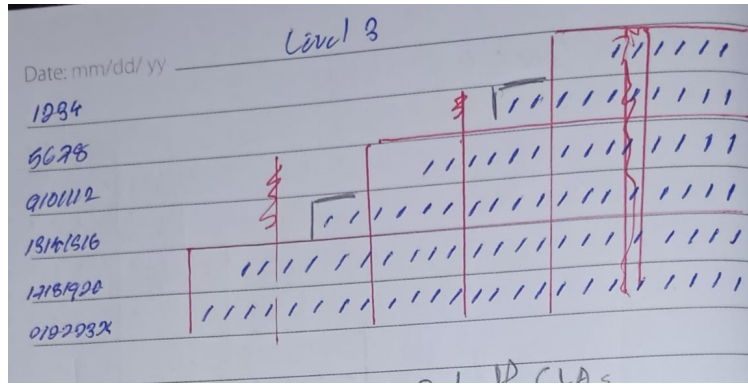


Figure 5: Level 3 - Adding consequently shifted versions of level 2 results

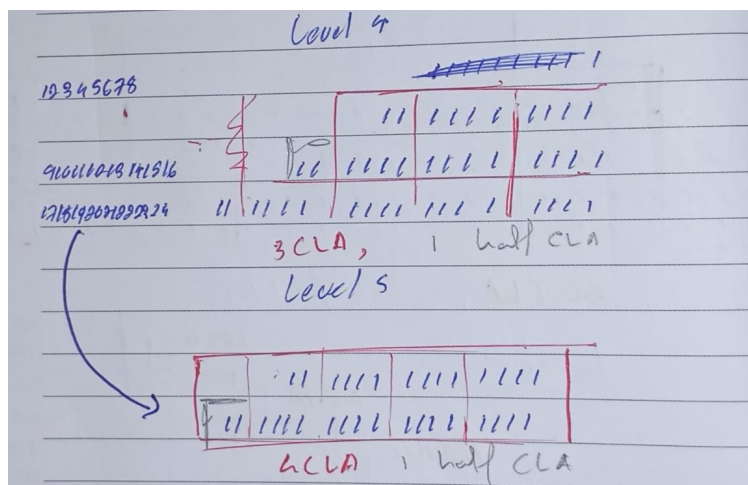


Figure 6: Level 4 and 5 - Adding consequently shifted versions of level 3 results

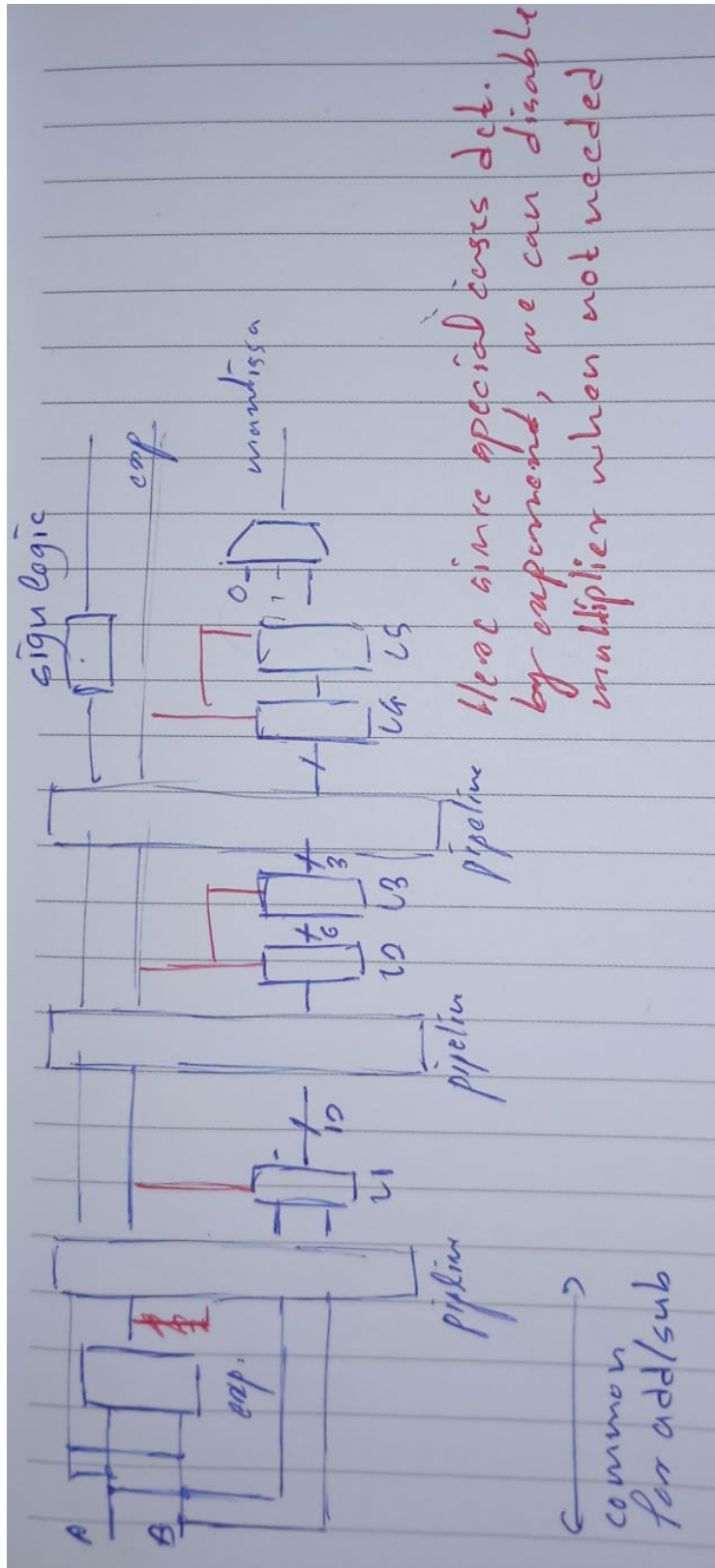


Figure 7: Overall design